LAPORAN PRAKTIKUM TUGAS AKHIR PENGEMBANGAN APLIKASI BERBASIS WEB
Modul 6: Generate Data Otomatis dan Authentication dengan Laravel



Universitas Telkom

Disusun Oleh:
Nama: Dryhus Dzacky Damingtyas.S
NIM: 707012530010
Kelas: D4SIKC-49-04
Mata Kuliah: Pengembangan Aplikasi Berbasis Web

UNIVERSITAS TELKOM
FAKULTAS ILMU TERAPAN
D4 SISTEM INFORMASI KOTACERDAS

Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami cara kerja authentication, session, hashing password, dan middleware pada Laravel.
2. Memasang Laravel Breeze dan memakai fitur register, login, serta logout yang dibuat otomatis.
3. Menelusuri alur register, login, logout, dan pemeriksaan middleware auth dari route sampai view.
4. Membuat akun pengguna dummy dengan seeder dan Faker, lalu memastikan password tersimpan dalam bentuk hash.
5. Membandingkan authentication yang dibuat dengan Breeze dan yang dibuat manual.

Alat dan Bahan

Praktikum ini dikerjakan pada laptop dengan spesifikasi berikut.

Perangkat Keras

- Laptop dengan prosesor 13th Gen Intel Core i7-13620H.
- RAM 16 GB (terbaca 15,7 GB).
- Sistem operasi Windows 11 Home Single Language (build 10.0.26200).

Perangkat Lunak

- XAMPP 8.2.12, di dalamnya terdapat Apache 2.4.58, MariaDB 10.4.32, PHP 8.2.12, dan phpMyAdmin 5.2.1.
- Composer 2.10.3 sebagai dependency manager PHP.
- Laravel Framework 12.70.0.
- Laravel Breeze 2.4.2 sebagai starter kit authentication.
- Node.js 24.11.1 dan NPM 11.6.2 untuk mengompilasi asset Breeze.
- Vite 7.3.7 sebagai build tool asset front-end.
- Visual Studio Code 1.142.0 sebagai teks editor.
- Google Chrome 155.0.8059.80 untuk menjalankan aplikasi di browser.
- Git 2.53.0 untuk version control.

• Database MySQL/MariaDB bernama db_modul6.

C. Langkah-Langkah Praktikum

Praktikum dikerjakan secara berurutan, mulai dari menyiapkan database sampai menguji alur authentication di browser. Setiap file disimpan sebelum lanjut ke langkah berikutnya. Project yang saya pakai adalah project Laravel baru di folder praktikum6, bukan project LaporBanjir dari Modul 4. Saya memilih project terpisah karena perintah breeze:install blade menimpa routes/web.php dan beberapa view, dan modul memang menyarankan membuat project baru atau membackup file lama.

C.1 Menyiapkan Database dan Project

Langkah pertama adalah menyalakan Apache dan MySQL pada XAMPP Control Panel. Setelah keduanya berjalan, saya membuka phpMyAdmin di http://localhost/phpmyadmin, memilih menu New, lalu membuat database baru bernama db_modul6. Database ini masih kosong karena tabelnya dibuat melalui migration.

Koneksi database diatur pada file .env di folder utama project. Bagian yang saya ubah adalah konfigurasi DB seperti berikut.

```
APP_NAME=Laravel
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_modul6
DB_USERNAME=root
DB_PASSWORD=

```

Nilai DB_CONNECTION diisi mysql karena Laravel versi terbaru secara bawaan memakai sqlite. DB_DATABASE diisi db_modul6, DB_USERNAME diisi root, dan DB_PASSWORD dibiarkan kosong mengikuti pengaturan default XAMPP. Setelah itu saya menjalankan php artisan migrate. Perintah ini membuat tabel users, password_reset_tokens, sessions, cache, jobs, dan tabel migrations yang mencatat migration yang sudah dijalankan.

Tabel users adalah tabel utama untuk authentication. Migration bawaannya membuat kolom id, name, email, email_verified_at, password, remember_token, created_at, dan updated_at. Kolom password bertipe varchar(255) dan nantinya diisi hash, bukan password asli. Potongan migration tabel users terlihat pada kode berikut.

```php
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->timestamp('email_verified_at')->nullable();
    $table->string('password');
    $table->rememberToken();
    $table->timestamps();
});

```

C.2 Instalasi Laravel Breeze

Laravel Breeze dipasang melalui Composer, lalu diinstal dengan tampilan Blade. Perintah yang saya jalankan adalah composer require laravel/breeze, php artisan breeze:install blade, npm install, npm run dev, dan php artisan migrate. Perintah breeze:install blade membuat route authentication, controller di folder app/Http/Controllers/Auth, view di folder resources/views/auth, layout, halaman profile, dan file pengujian secara otomatis.

Breeze memakai Tailwind CSS dan Vite. Karena itu Node.js dan NPM harus tersedia, lalu npm run dev harus tetap berjalan selama pengembangan. Pada praktikum ini Vite berjalan di http://localhost:5173 dan Laravel dijalankan dengan php artisan serve di http://127.0.0.1:8000. Kalau proses Vite berhenti, Laravel tidak menemukan manifest asset dan halaman bisa menampilkan galat Vite manifest not found.

C.3 Route dan Middleware Authentication

Route utama aplikasi ada di routes/web.php. Halaman / menampilkan welcome, /dashboard menampilkan dashboard, dan route profile dimasukkan ke dalam grup middleware auth. Di bagian bawah file ada require __DIR__.'/auth.php' yang memuat seluruh route authentication dari file terpisah. Isi routes/web.php pada project ini adalah sebagai berikut.

```php
<?php

use App\Http\Controllers\ProfileController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');
});

require __DIR__.'/auth.php';

```

Route /dashboard memakai middleware ['auth', 'verified'] dan diberi nama dashboard. Middleware auth memeriksa apakah pengguna sudah login. Middleware verified hanya berpengaruh kalau model User mengimplementasikan MustVerifyEmail. Pada project ini baris use Illuminate\Contracts\Auth\MustVerifyEmail masih dikomentari, jadi pengguna baru tetap bisa membuka dashboard tanpa verifikasi email. Route profile memakai middleware auth tanpa verified.

File routes/auth.php memisahkan route menjadi dua grup. Grup guest berisi register, login, forgot-password, dan reset-password. Grup auth berisi verifikasi email, konfirmasi password, update password, dan logout. Potongan route utamanya adalah sebagai berikut.

```php
<?php

use App\Http\Controllers\Auth\AuthenticatedSessionController;
use App\Http\Controllers\Auth\RegisteredUserController;
use Illuminate\Support\Facades\Route;

Route::middleware('guest')->group(function () {
    Route::get('register', [RegisteredUserController::class, 'create'])
        ->name('register');

    Route::post('register', [RegisteredUserController::class, 'store']);

    Route::get('login', [AuthenticatedSessionController::class, 'create'])
        ->name('login');

    Route::post('login', [AuthenticatedSessionController::class, 'store']);
});

Route::middleware('auth')->group(function () {
    Route::post('logout', [AuthenticatedSessionController::class, 'destroy'])
        ->name('logout');
});

```

Middleware guest mencegah pengguna yang sudah login membuka halaman register dan login. Middleware auth mengarahkan pengguna yang belum login ke route bernama login. Karena itu route login wajib punya nama login. Logout memakai method POST dan tidak memakai GET supaya proses keluar tidak bisa dipicu hanya dengan membuka sebuah tautan.

C.4 Controller Register dan Login

Controller register bawaan Breeze adalah RegisteredUserController. Method create() menampilkan view auth.register, sedangkan method store() memvalidasi input, membuat user baru, dan langsung melakukan login. Validasi memeriksa nama, email yang unik, dan password yang dikonfirmasi. Password minimal 8 karakter mengikuti Rules\Password::defaults(). Potongan method store() adalah sebagai berikut.

```php
public function store(Request $request): RedirectResponse
{
    $request->validate([
        'name' => ['required', 'string', 'max:255'],
        'email' => ['required', 'string', 'lowercase', 'email', 'max:255',
'unique:'.User::class],
        'password' => ['required', 'confirmed', Rules\Password::defaults()],
    ]);

    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->password),
    ]);

    event(new Registered($user));

    Auth::login($user);

    return redirect(route('dashboard', absolute: false));
}
```

User::create() menyimpan nama, email, dan password. Password diubah menjadi hash lewat Hash::make() sebelum disimpan. Setelah itu event(new Registered($user)) dipicu, Auth::login($user) membuat session login, dan pengguna diarahkan ke dashboard. Karena model User belum mengimplementasikan MustVerifyEmail, email verifikasi tidak diwajibkan pada project ini.

Controller login bawaan Breeze adalah AuthenticatedSessionController. Method create() menampilkan view auth.login. Method store() memanggil LoginRequest untuk memeriksa kredensial, membuat ulang session, lalu mengarahkan pengguna ke dashboard. Method destroy() menghapus status login, mengosongkan session, mengganti token CSRF, dan mengarahkan pengguna ke halaman utama. Potongan kedua method itu adalah sebagai berikut.

```php
public function store(LoginRequest $request): RedirectResponse
{
    $request->authenticate();

    $request->session()->regenerate();

    return redirect()->intended(route('dashboard', absolute: false));
}

public function destroy(Request $request): RedirectResponse
{
    Auth::guard('web')->logout();

    $request->session()->invalidate();

    $request->session()->regenerateToken();

    return redirect('/');
}

```

Pemeriksaan kredensial ada di LoginRequest. Method rules() memastikan email dan password diisi. Method authenticate() memakai Auth::attempt() dengan email, password, dan checkbox remember. Kalau gagal, RateLimiter mencatat percobaan dan ValidationException menampilkan pesan auth.failed. Kalau berhasil, RateLimiter dibersihkan. Potongan method rules() dan authenticate() adalah sebagai berikut.

```php
public function rules(): array
{
    return [
        'email' => ['required', 'string', 'email'],
        'password' => ['required', 'string'],
    ];
}

public function authenticate(): void
{
    $this->ensureIsNotRateLimited();

    if (! Auth::attempt($this->only('email', 'password'), $this->boolean('remember'))) {
        RateLimiter::hit($this->throttleKey());

        throw ValidationException::withMessages([
            'email' => trans('auth.failed'),
        ]);
    }

    RateLimiter::clear($this->throttleKey());
}

```

Model User menyimpan daftar kolom yang boleh diisi, menyembunyikan password dan remember_token dari serialisasi, serta mengubah kolom password menjadi hash lewat cast hashed. Potongan pengaturannya adalah sebagai berikut.

```php
protected $fillable = [
    'name',
    'email',
    'password',
];

protected $hidden = [
    'password',
    'remember_token',
];

protected function casts(): array
{
    return [
        'email_verified_at' => 'datetime',
        'password' => 'hashed',
    ];
}

```

C.5 View dan Layout

Breeze memakai dua layout utama. Layout guest dipakai untuk halaman login dan register, sedangkan layout app dipakai untuk halaman yang sudah login seperti dashboard dan profile. Layout app memuat navigation, header opsional, dan slot konten. Potongan layouts/app.blade.php adalah sebagai berikut.

```
<!DOCTYPE html>
<html lang="{{ str_replace('_', '-', app()->getLocale()) }}">
    <head>
        <meta charset="utf-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <meta name="csrf-token" content="{{ csrf_token() }}">

        <title>{{ config('app.name', 'Laravel') }}</title>

        <!-- Fonts -->
        <link rel="preconnect" href="https://fonts.bunny.net">
        <link href="https://fonts.bunny.net/css?family=figtree:400,500,600&display=swap"
rel="stylesheet" />

        <!-- Scripts -->
        @vite(['resources/css/app.css', 'resources/js/app.js'])
    </head>
    <body class="font-sans antialiased">
        <div class="min-h-screen bg-gray-100">
            @include('layouts.navigation')

            <!-- Page Heading -->
            @isset($header)
                <header class="bg-white shadow">
                    <div class="max-w-7xl mx-auto py-6 px-4 sm:px-6 lg:px-8">
                        {{ $header }}
                    </div>
                </header>
            @endisset

            <!-- Page Content -->
            <main>
                {{ $slot }}
            </main>
        </div>
    </body>
</html>
```

Halaman register memakai komponen Blade untuk label, input, pesan galat, dan tombol. Directive @csrf menambahkan token keamanan. Atribut name pada input harus sama dengan nama kolom yang divalidasi. Nilai lama diisi kembali lewat old() supaya pengguna tidak perlu mengetik ulang ketika validasi gagal. Potongan form register adalah sebagai berikut.

```
<form method="POST" action="{{ route('register') }}">
    @csrf

    <div>
        <x-input-label for="name" :value="__('Name')" />
        <x-text-input id="name" class="block mt-1 w-full" type="text" name="name"
:value="old('name')" required autofocus autocomplete="name" />
        <x-input-error :messages="$errors->get('name')" class="mt-2" />
    </div>

    <div class="mt-4">
        <x-input-label for="email" :value="__('Email')" />
        <x-text-input id="email" class="block mt-1 w-full" type="email" name="email"
:value="old('email')" required autocomplete="username" />
        <x-input-error :messages="$errors->get('email')" class="mt-2" />
    </div>

    <div class="mt-4">
        <x-input-label for="password" :value="__('Password')" />
        <x-text-input id="password" class="block mt-1 w-full" type="password"
name="password" required autocomplete="new-password" />
        <x-input-error :messages="$errors->get('password')" class="mt-2" />
    </div>

    <div class="mt-4">
        <x-input-label for="password_confirmation" :value="__('Confirm Password')" />
        <x-text-input id="password_confirmation" class="block mt-1 w-full" type="password"
name="password_confirmation" required autocomplete="new-password" />
        <x-input-error :messages="$errors->get('password_confirmation')" class="mt-2" />
    </div>

    <div class="flex items-center justify-end mt-4">
        <x-primary-button class="ms-4">
            {{ __('Register') }}
        </x-primary-button>
    </div>
</form>

```

Halaman login memakai pola yang sama, ditambah checkbox Remember me dan tautan lupa password. Potongan form login adalah sebagai berikut.

```
<form method="POST" action="{{ route('login') }}">
    @csrf

    <div>
        <x-input-label for="email" :value="__('Email')" />
        <x-text-input id="email" class="block mt-1 w-full" type="email" name="email" :value="old('email')" required autofocus autocomplete="username" />
        <x-input-error :messages="$errors->get('email')" class="mt-2" />
    </div>

    <div class="mt-4">
        <x-input-label for="password" :value="__('Password')" />
        <x-text-input id="password" class="block mt-1 w-full" type="password" name="password" required autocomplete="current-password" />
        <x-input-error :messages="$errors->get('password')" class="mt-2" />
    </div>

    <div class="block mt-4">
        <label for="remember_me" class="inline-flex items-center">
            <input id="remember_me" type="checkbox" class="rounded border-gray-300 text-indigo-600 shadow-sm focus:ring-indigo-500" name="remember">
            <span class="ms-2 text-sm text-gray-600">{{ __('Remember me') }}</span>
        </label>
    </div>

    <div class="flex items-center justify-end mt-4">
        <x-primary-button class="ms-3">
            {{ __('Log in') }}
        </x-primary-button>
    </div>
</form>

```

Pesan galat validasi muncul di bawah input yang bermasalah. Pada halaman register, password yang kurang dari 8 karakter dan konfirmasi yang tidak sama akan menampilkan dua pesan sekaligus. Setelah data diperbaiki, form dikirim ke route register dan pengguna langsung masuk ke dashboard.

C.6 Seeder User dengan Faker

Setelah tabel users dibuat, isinya masih kosong. Untuk mengisi akun contoh, saya membuat seeder dengan perintah php artisan make:seeder UserSeeder. Seeder ini memakai Faker untuk menghasilkan nama dan email acak, lalu menyimpan password dalam bentuk hash. Potongan isi UserSeeder.php adalah sebagai berikut.

```php
$faker = Faker::Create();

for ($i = 0; $i < 10; $i++) {
    DB::table('users')->insert([
        'name' => $faker->name,
        'email' => $faker->unique()->safeEmail,
        'password' => Hash::make('password'),
        'created_at' => now(),
        'updated_at' => now(),
    ]);
}

DB::table('users')->insert([
    'name' => 'Admin',
    'email' => 'admin@example.com',
    'password' => Hash::make('admin123'),
    'created_at' => now(),
    'updated_at' => now(),
]);

```

Pada project ini Faker dipanggil dengan Faker::Create() tanpa locale, sehingga nama dan email dummy yang muncul berbahasa Inggris. Perulangan for ($i = 0; $i < 10; $i++) membuat 10 akun dummy dengan password password. Setelah itu seeder menambahkan satu akun admin dengan email admin@example.com dan password admin123. Seluruh password disimpan lewat Hash::make(), jadi isi kolom password di database berupa hash bcrypt, bukan teks asli.

DatabaseSeeder.php bawaan project masih berisi User::factory()->create() untuk Test User dan belum memanggil UserSeeder. Karena itu saya menjalankan seeder secara langsung dengan php artisan db:seed --class=UserSeeder. Kalau hanya php artisan db:seed yang dijalankan, yang dibuat adalah akun Test User, bukan 10 akun dummy dan akun admin.

C.7 Menjalankan Aplikasi dan Menguji Alur

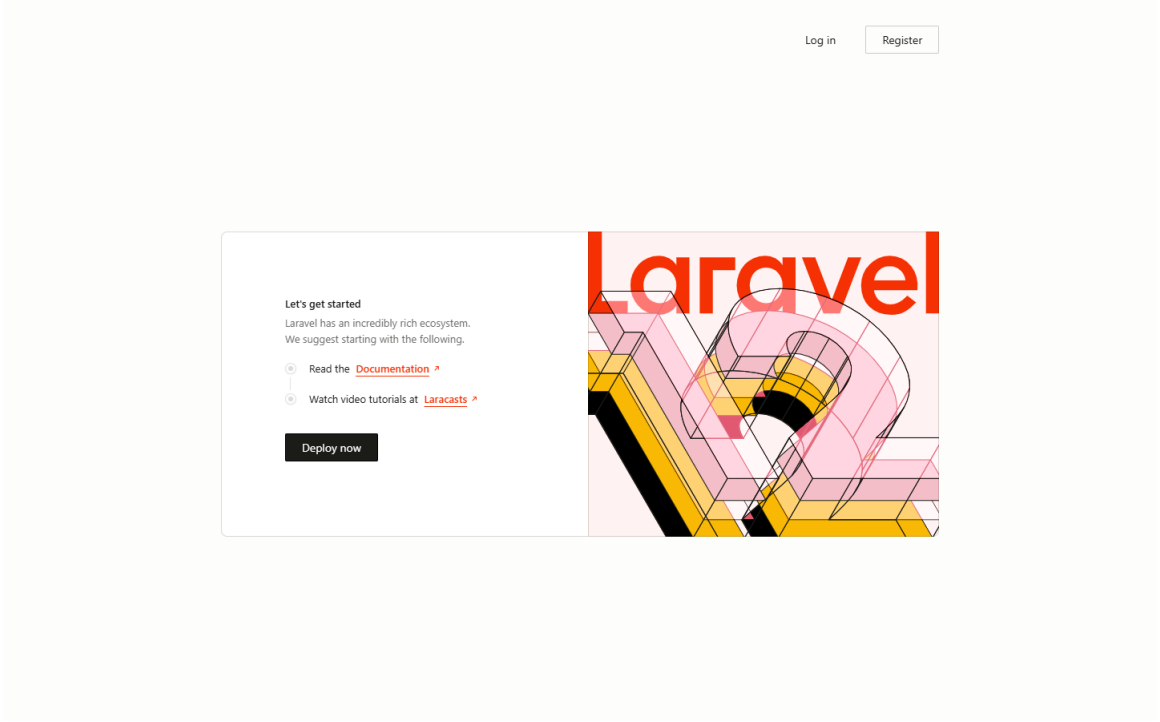
Aplikasi dijalankan dengan dua terminal. Terminal pertama menjalankan npm run dev untuk Vite, terminal kedua menjalankan php artisan serve untuk Laravel. Server berjalan di http://127.0.0.1:8000. Apache dan MySQL pada XAMPP juga harus tetap berjalan karena database db_modul6 dipakai oleh aplikasi.

Pengujian dimulai dengan membuka /dashboard tanpa login. Middleware auth mengarahkan saya ke /login. Setelah itu saya membuka /register, mengisi nama, email, password, dan konfirmasi password. Saya sempat mengisi password pendek dan konfirmasi yang berbeda untuk melihat pesan validasi. Setelah data diperbaiki, registrasi berhasil dan saya diarahkan ke dashboard.

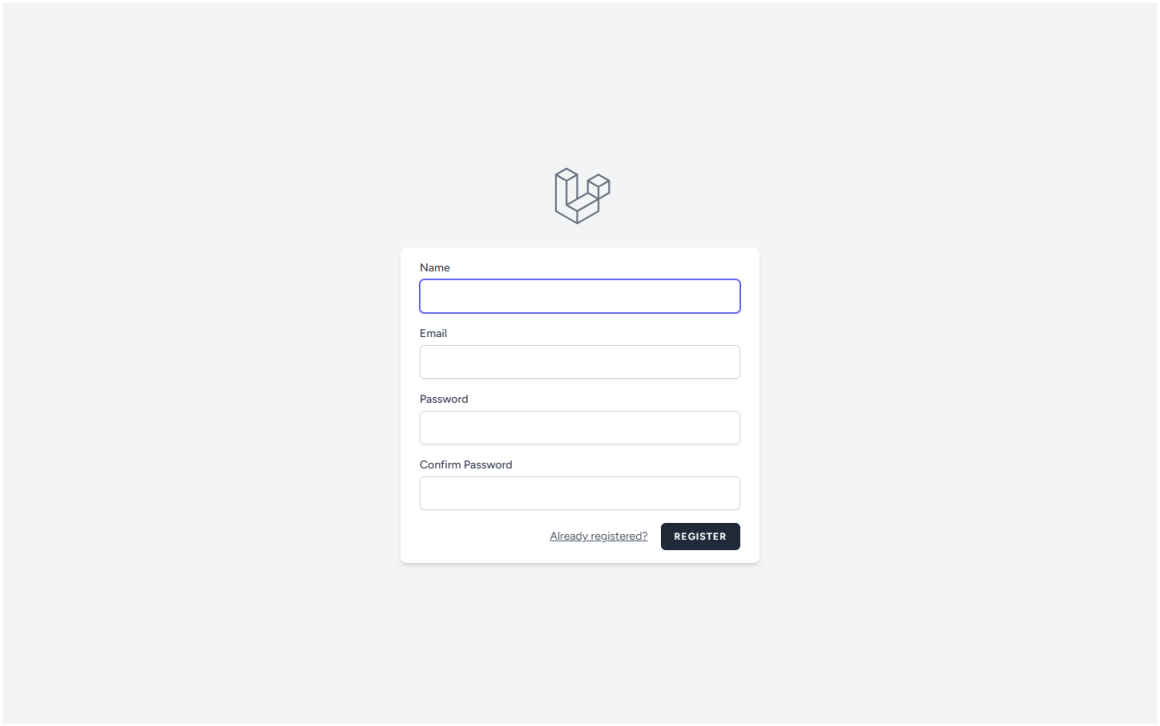
Dari dashboard saya membuka menu pengguna, melihat halaman profile, lalu logout. Saya login kembali dengan email dan password yang benar, lalu mencoba login dengan password salah untuk melihat pesan galat. Terakhir saya membuka tabel users di phpMyAdmin untuk memastikan akun hasil register dan hasil seeder tersimpan, serta kolom password berisi hash. Hasil setiap pengujian ada pada bagian D.

D. Hasil dan Dokumentasi Program

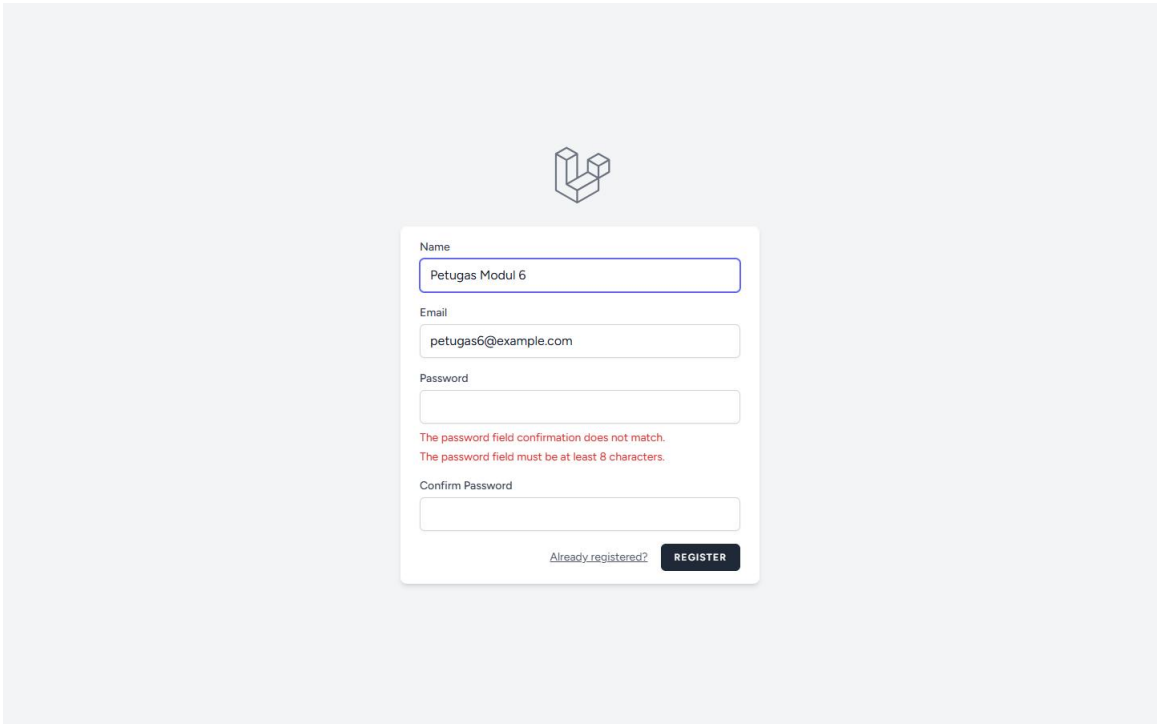
Berikut tangkapan layar dari aplikasi yang sudah dijalankan dan dari phpMyAdmin.



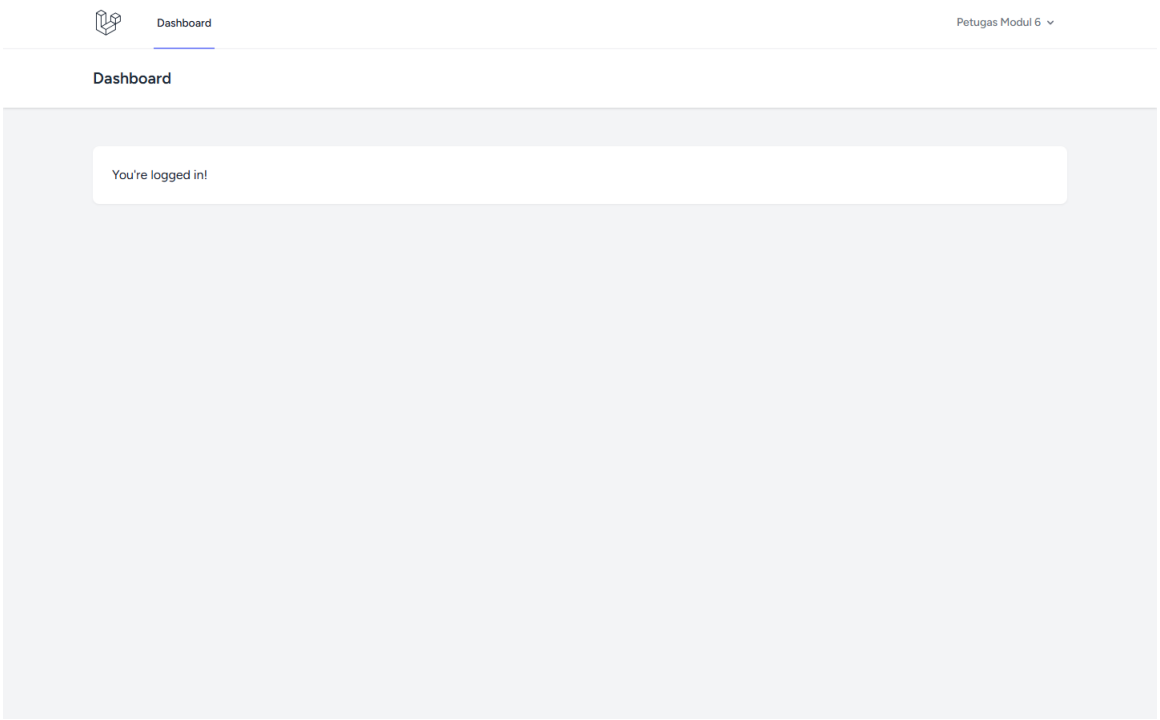
Gambar 1. Halaman awal aplikasi dengan menu Log in dan Register.



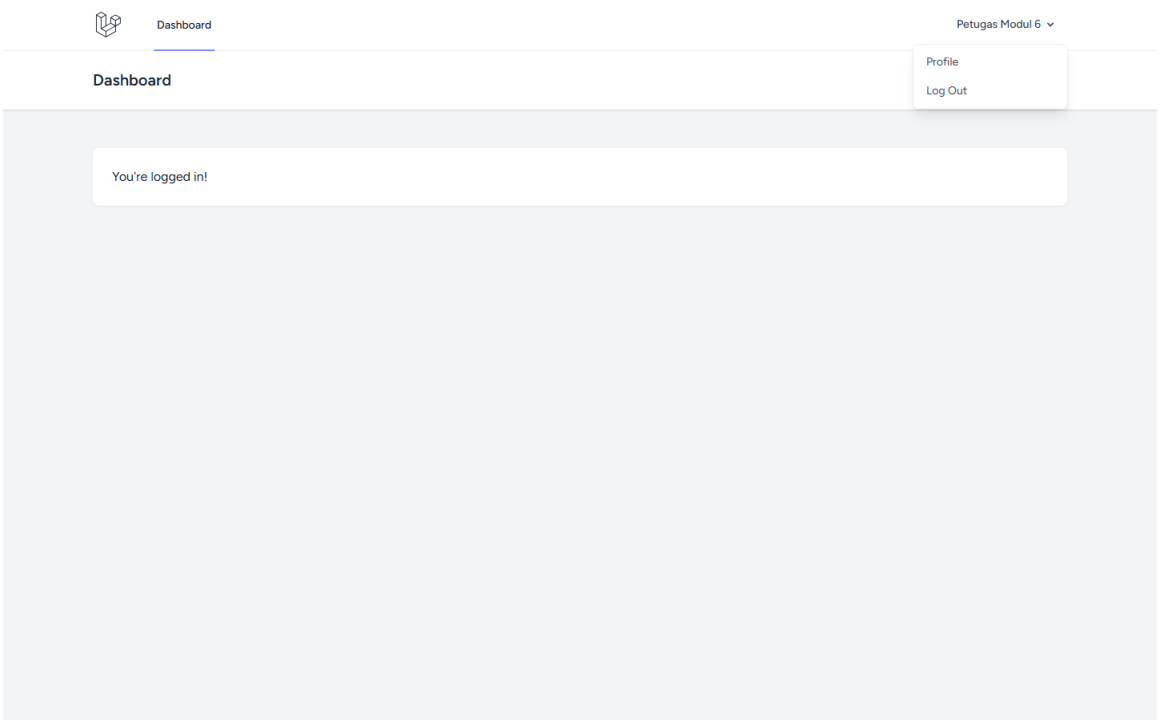
Gambar 2. Halaman register sebelum diisi.



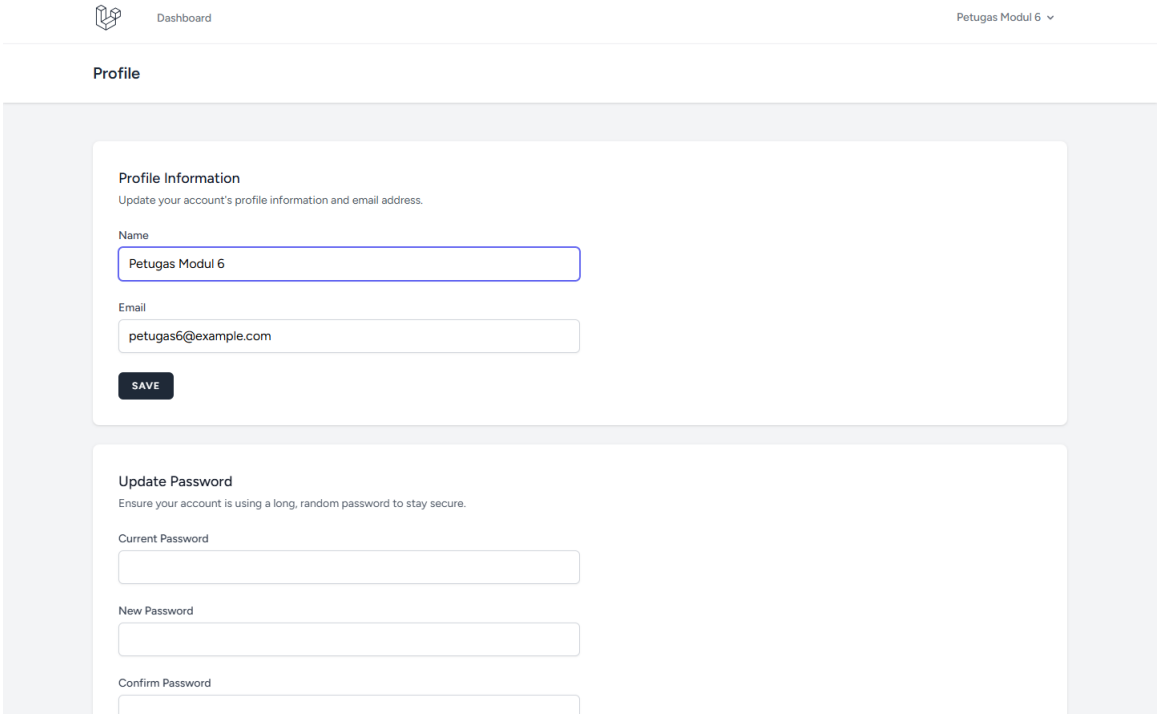
Gambar 3. Pesan galat validasi saat password kurang dari 8 karakter dan konfirmasi tidak sama.



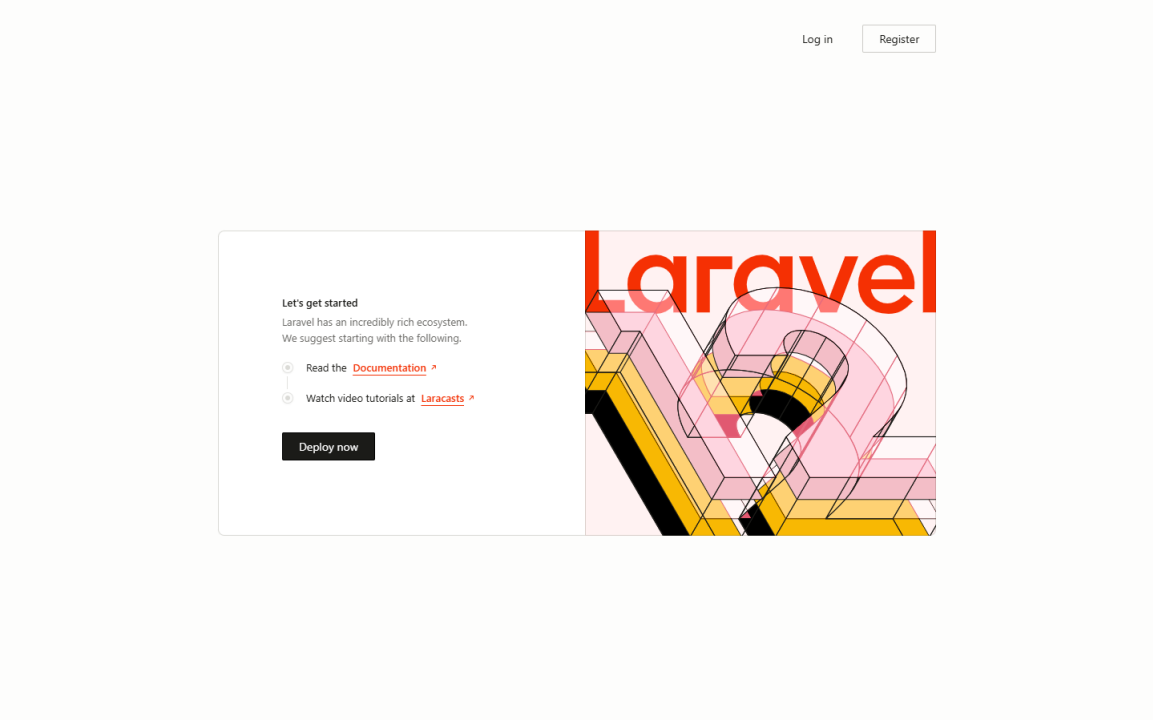
Gambar 4. Halaman dashboard setelah registrasi berhasil. Nama pengguna muncul di pojok kanan atas.



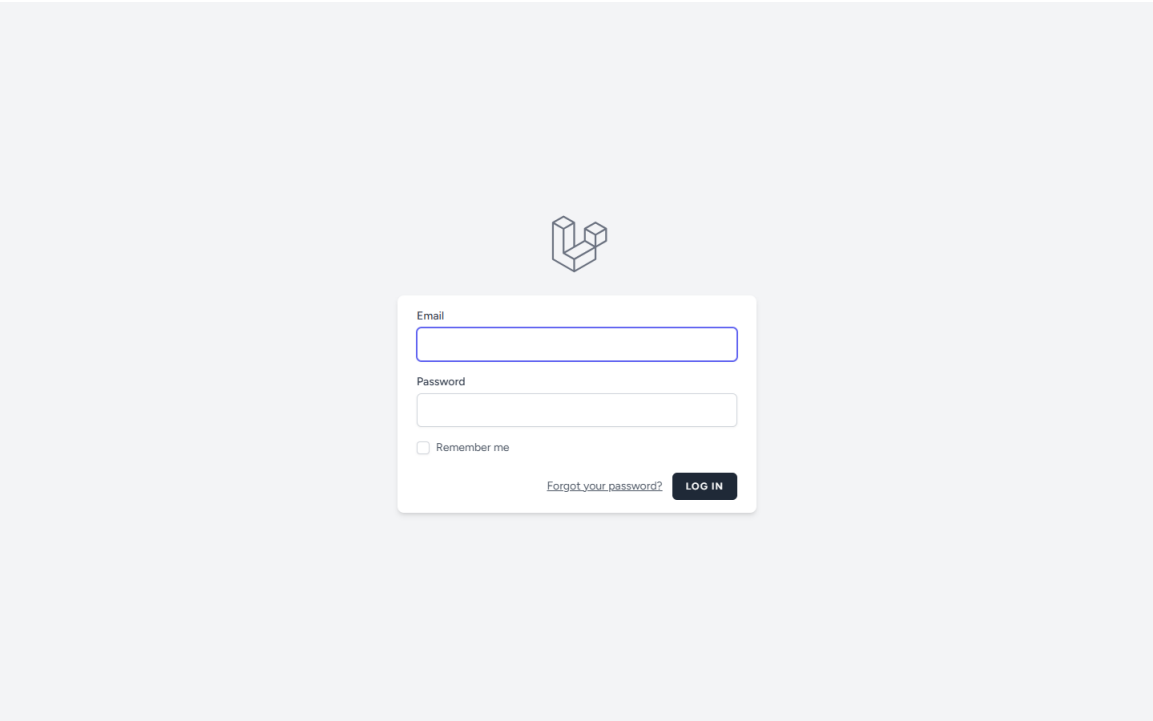
Gambar 5. Menu dropdown pengguna yang berisi Profile dan Log Out.



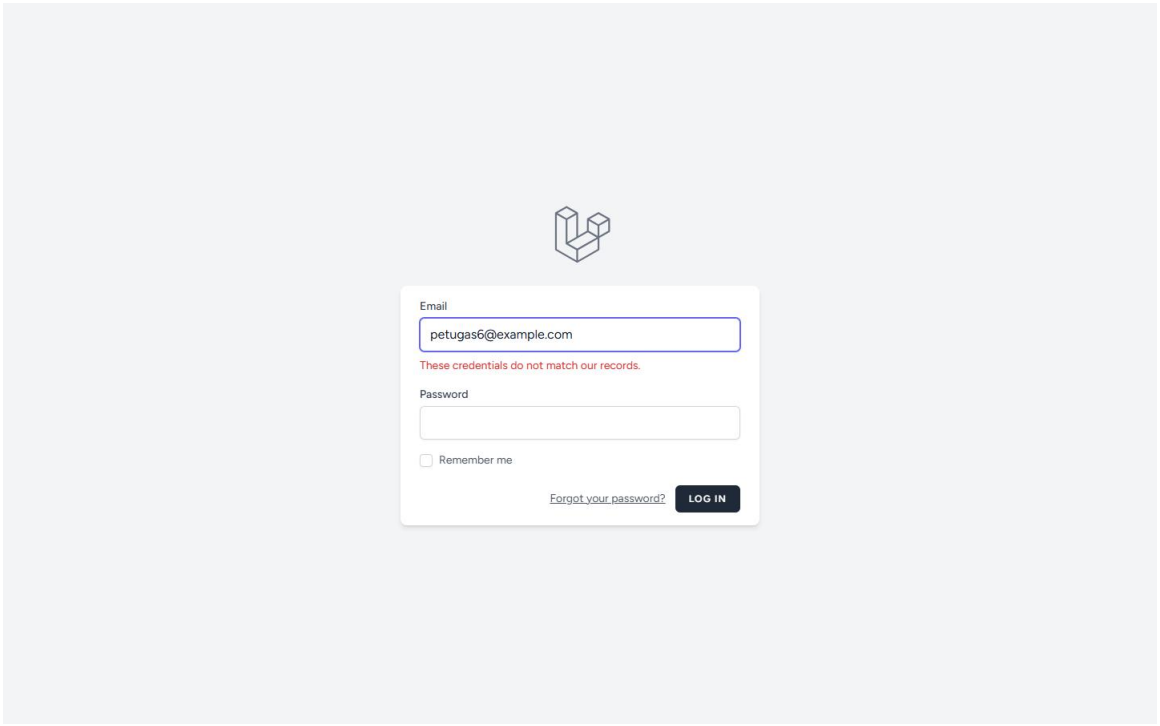
Gambar 6. Halaman profile untuk mengubah data akun.



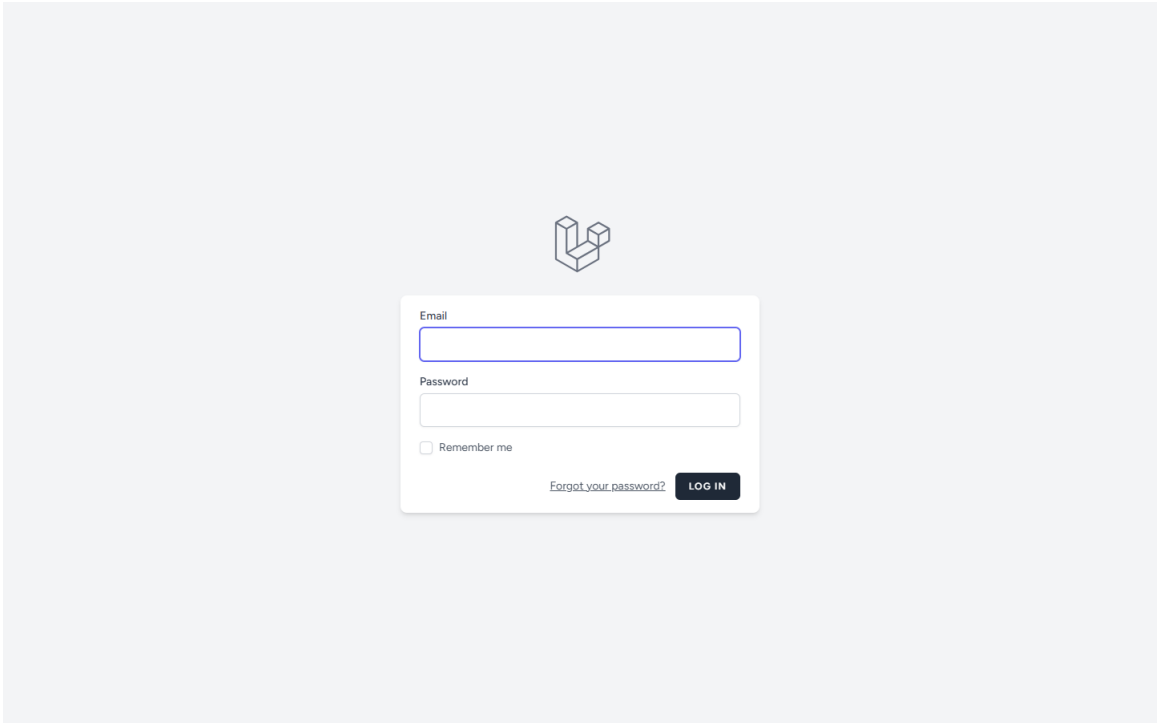
Gambar 7. Halaman awal setelah logout. Menu Log in dan Register muncul kembali.



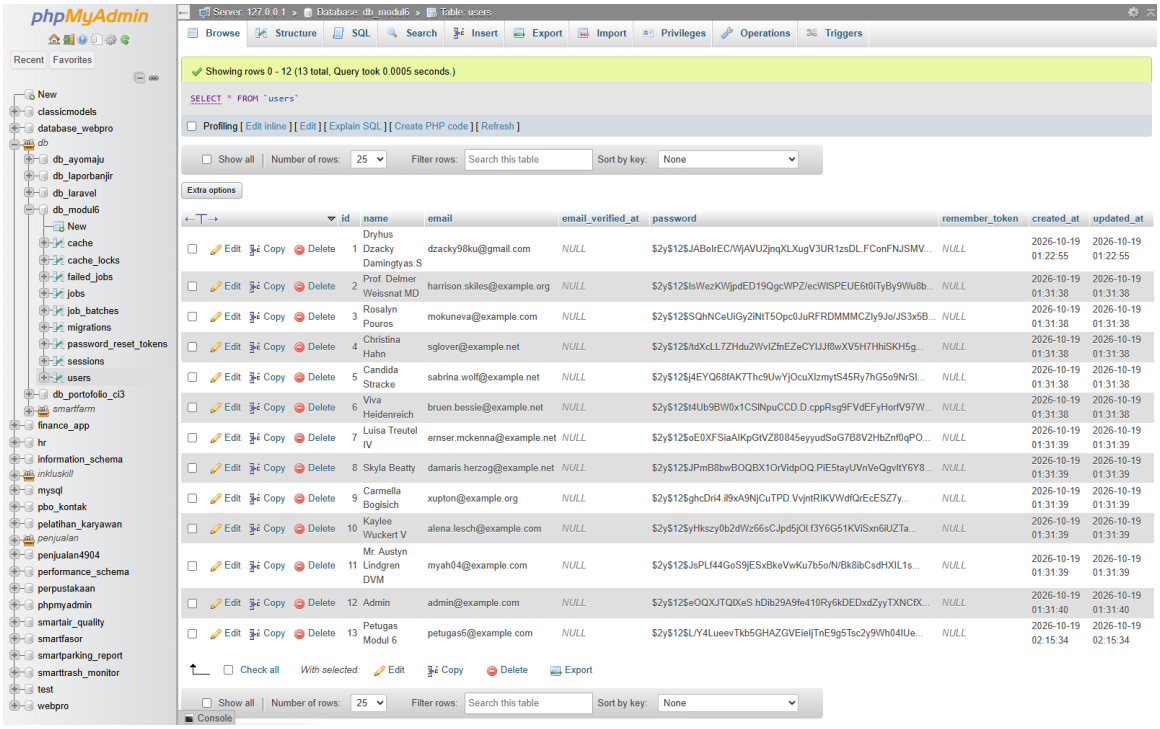
Gambar 8. Halaman login.



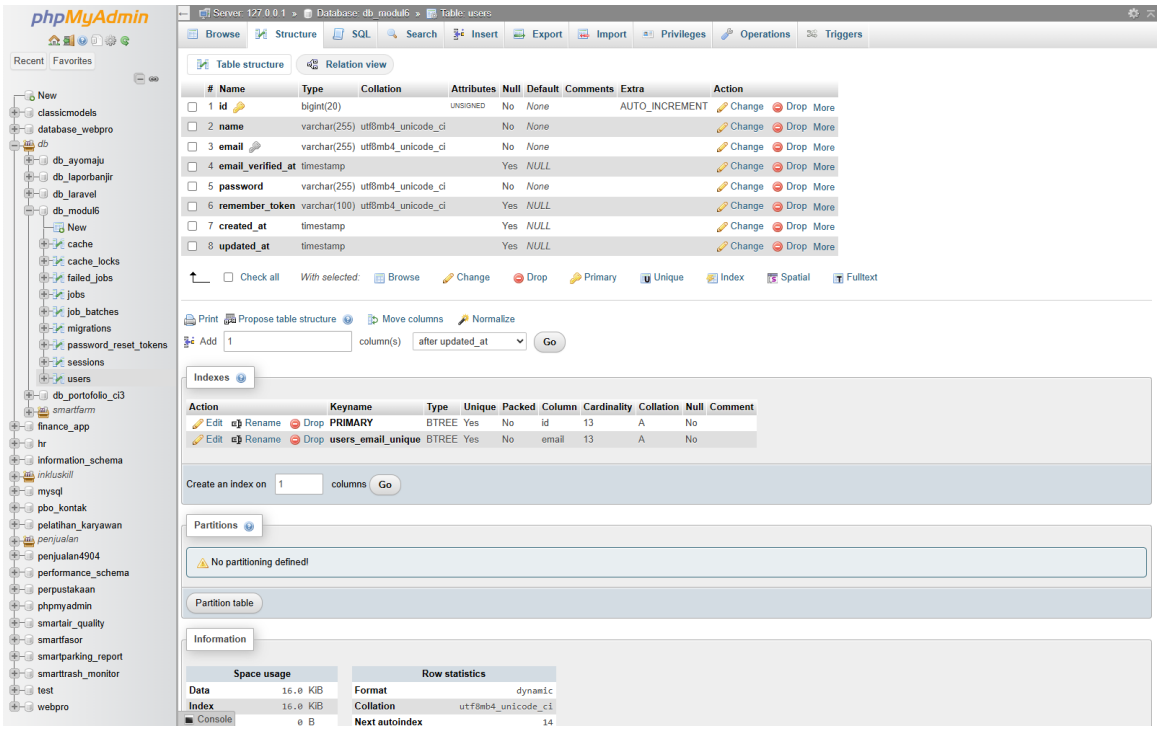
Gambar 9. Pesan galat These credentials do not match our records. saat password salah.



Gambar 10. Halaman login yang muncul setelah /dashboard dibuka tanpa login.



Gambar 11. Tabel users di phpMyAdmin setelah register dan seeder. Kolom password berisi hash dengan awalan $2y$12$.



Gambar 12. Struktur tabel users. Kolom password bertipe varchar(255).

E. Analisis dan Pembahasan

E.1 Peran Authentication, Session, dan Middleware

Authentication memastikan bahwa pengguna yang mengakses aplikasi adalah orang yang ia akui. Pada project ini proses itu terjadi di halaman login. Setelah email dan password cocok, Laravel menyimpan status login di session. Browser hanya menyimpan cookie session, sedangkan data session disimpan di server. Project ini memakai SESSION_DRIVER=database, sehingga data session tersimpan di tabel sessions.

Setiap request berikutnya membawa cookie session. Middleware auth membaca session itu dan memeriksa apakah pengguna sudah login. Kalau sudah, request diteruskan ke controller atau view. Kalau belum, pengguna diarahkan ke route login. Middleware guest bekerja sebaliknya, yaitu mencegah pengguna yang sudah login membuka halaman register dan login. Pada project ini /dashboard dilindungi auth dan verified, sedangkan /profile dilindungi auth.

Middleware verified tidak otomatis mewajibkan verifikasi email. Middleware itu hanya memeriksa apakah pengguna sudah memverifikasi email jika model User mengimplementasikan MustVerifyEmail. Pada project ini interface tersebut masih dikomentari, jadi pengguna baru langsung bisa membuka dashboard. Perilaku ini sesuai bawaan Breeze dan tidak mengubah alur login.

E.2 Hashing Password

Password tidak disimpan dalam bentuk teks asli. Saat register, RegisteredUserController memanggil Hash::make() sebelum menyimpan password. Saat login, Auth::attempt() mencocokkan password yang diketik dengan hash di database memakai Hash::check(). Karena itu password asli tidak pernah tersimpan dan tidak bisa dibaca langsung dari tabel users.

Hash yang muncul di phpMyAdmin berawalan $2y$12$. Awalan itu menunjukkan algoritma bcrypt dengan cost 12. Bcrypt menambahkan salt dan sengaja dibuat lambat supaya percobaan menebak password secara massal lebih sulit. Model User juga menyembunyikan password dan remember_token lewat properti $hidden, serta memakai cast hashed untuk kolom password. Dengan pengaturan ini, password tetap aman walaupun isi database dilihat orang lain.

E.3 Perbedaan Breeze dan Authentication Manual

Breeze menyiapkan hampir semua kebutuhan authentication secara otomatis. Route register, login, logout, forgot-password, reset-password, verifikasi email, dan profile sudah dibuat.

Controller, view, layout, dan file pengujian juga sudah tersedia. Pekerjaan yang tersisa adalah menyesuaikan database, menjalankan migration, mengompilasi asset, dan menguji alurnya. Cara ini cepat dipakai untuk prototipe atau project yang ingin segera punya fitur login.

Authentication manual memerlukan lebih banyak file yang ditulis sendiri. Menurut modul, kita perlu membuat RegisterController, LoginController, dan HomeController, lalu menulis route, layout, view register, view login, view dashboard, dan view profile. Proses login memakai Auth::attempt(), proses logout memakai Auth::logout(), dan password di-hash dengan Hash::make(). Halaman yang dilindungi dimasukkan ke grup middleware auth.

Perbedaan utamanya ada pada jumlah kode dan tingkat kendali. Breeze lebih cepat dan sudah mencakup fitur tambahan, tetapi proses di balik layar kurang terlihat karena kode dibuat otomatis. Versi manual lebih banyak menulis kode dan lebih mudah salah, misalnya route login belum diberi nama, password disimpan tanpa hash, atau token CSRF tidak dikirim. Sebaliknya, versi manual membuat alur request, controller, session, dan middleware lebih jelas karena semuanya ditulis sendiri.

E.4 Alur Register, Login, dan Logout

Alur register dimulai dari GET /register. Route memanggil RegisteredUserController@create dan menampilkan form. Saat form dikirim dengan POST, method store() memvalidasi nama, email, dan password. Kalau valid, User::create() menyimpan akun dengan password hash, event Registered dipicu, Auth::login() membuat session, dan pengguna diarahkan ke /dashboard.

Alur login dimulai dari GET /login yang menampilkan form. Saat form dikirim dengan POST, LoginRequest memvalidasi input lalu memanggil Auth::attempt(). Kalau kredensial cocok, session dibuat ulang dengan regenerate() untuk mencegah session fixation, lalu pengguna diarahkan ke halaman yang sebelumnya ingin dibuka atau ke dashboard. Kalau gagal, pengguna kembali ke halaman login dengan pesan galat.

Alur logout dimulai dari tombol Log Out di menu pengguna. Tombol itu mengirim POST /logout. AuthenticatedSessionController@destroy memanggil Auth::logout(), mengosongkan session dengan invalidate(), mengganti token CSRF dengan regenerateToken(), lalu mengarahkan pengguna ke halaman utama. Setelah itu halaman /dashboard tidak bisa dibuka lagi sampai pengguna login kembali.

E.5 Kendala dan Penyelesaian

Ada dua kendala yang saya temui selama praktikum.

1. Perintah php artisan make:RegisterController tidak dikenali. Laravel menampilkan pesan Command make:RegisterController is not defined beserta daftar perintah yang mirip. Penyebabnya adalah nama perintah generator yang salah. Perintah yang benar adalah php artisan make:controller Auth/RegisterController. Pada project ini controller register sudah dibuat otomatis oleh Breeze, jadi perintah manual itu tidak diperlukan lagi.
2. Perintah npm diblokir oleh kebijakan eksekusi PowerShell. Muncul pesan npm.ps1 cannot be loaded because running scripts is disabled on this system. Saya menjalankan Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass pada terminal yang dipakai, lalu npm bisa dijalankan. Alternatif lain adalah memakai npm.cmd atau cmd /c npm run dev. Pada pengujian, Vite dijalankan lewat npm.cmd dan tetap hidup selama pengambilan tangkapan layar.

E.6 Catatan Perbedaan dengan Modul

Ada beberapa perbedaan antara project ini dan contoh pada modul. Perbedaan ini saya catat supaya hasil praktikum tidak dibaca sebagai hal yang sama persis dengan contoh.

1. Project yang dipakai adalah project Breeze terpisah di folder praktikum6, bukan project LaporBanjir dari Modul 4. Karena itu halaman yang dilindungi pada laporan ini adalah /dashboard dan /profile, bukan halaman daftar laporan. Pemisahan project dilakukan karena Breeze menimpa routes/web.php dan beberapa view.
2. UserSeeder memakai Faker::Create() tanpa locale id_ID dan membuat 10 akun dummy, bukan 5 akun seperti contoh modul. Akibatnya nama dan email dummy yang muncul berbahasa Inggris. Jumlah dan locale ini tidak memengaruhi proses login karena yang diuji adalah email, password, dan hash di database.
3. DatabaseSeeder.php belum memanggil UserSeeder. Seeder dijalankan langsung dengan php artisan db:seed --class=UserSeeder. Kalau ingin mengikuti struktur modul, UserSeeder bisa ditambahkan ke array $this->call([...]) di DatabaseSeeder, lalu seeder dijalankan dengan php artisan db:seed.

F. Kesimpulan

1. Laravel Breeze 2.4.2 berhasil dipasang dan menyediakan route, controller, view, serta layout untuk register, login, logout, reset password, verifikasi email, dan profile.
2. Alur authentication berjalan dengan baik. Register menyimpan akun dengan password hash, login memeriksa kredensial lewat Auth::attempt(), session menyimpan status login, dan logout menghapus session.
3. Middleware auth melindungi /dashboard dan /profile. Pengguna yang belum login diarahkan ke /login, sedangkan middleware guest mencegah pengguna yang sudah login membuka halaman register dan login.
4. Seeder UserSeeder berhasil membuat 10 akun dummy dan 1 akun admin. Password tersimpan sebagai hash bcrypt dengan awalan $2y$12$, dan tabel users di phpMyAdmin menunjukkan bukti tersebut.
5. Breeze dan authentication manual memakai konsep session, hashing, dan middleware yang sama. Bedanya, Breeze menyiapkan route, controller, dan view secara otomatis, sedangkan versi manual menuntut semuanya ditulis sendiri.
6. Praktikum ini juga menunjukkan pentingnya menjaga konfigurasi database, menjalankan Vite selama pengembangan, dan memastikan seeder dipanggil dari DatabaseSeeder atau dijalankan dengan --class.

G. Link Repository Github

Kode program lengkap praktikum ini dapat diakses pada repositori GitHub berikut:

https://github.com/Dryhus21/PABW.git